UNIT II

Functions and Classes & Objects in C++

ANSWERED QUESTION BANK

Debugging • Corrected C++ • Pseudo-Code • Dry Runs • Quick Revision

Section A – Debugging Questions

Question 1: Dangling Reference in Return-by-Reference

Errors and explanation

- Line 97 (return result) – Undefined behavior / dangling reference. result is a local automatic variable stored in the function activation record. Its lifetime ends when getMaximumScaled returns, so returning int& refers to an object that no longer exists.
- Line 109 and Line 117 – Runtime undefined behavior. danglingRef binds to the invalid returned reference; reading it may show garbage, crash, or appear to work unpredictably.
- getArrayElement is safe for the shown use because arr refers to the caller-owned array. The array remains alive in main while the returned element reference is used. The function must still receive a valid index and the array must outlive the reference.

Corrected complete C++ code

```
#include <iostream>

int getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    return maxVal * scaleFactor; // Return by value
}

int& getArrayElement(int arr[], int index) {
    return arr[index]; // Refers to caller-owned array element
}

int main() {
    int x = 10, y = 20;

    int scaledValue = getMaximumScaled(x, y, 2);
    std::cout << "Scaled Max: " << scaledValue << std::endl;

    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999;
    std::cout << "Updated Array[1]: " << data[1] << std::endl;

    std::cout << "Scaled Value Again: " << scaledValue << std::endl;
    return 0;
}
```

Dry run

Step	Operation	Result
1	x = 10, y = 20; maximum = 20	20
2	20 × scaleFactor 2	scaledValue = 40
3	data initially {100, 200, 300}	data[1] = 200
4	getArrayElement(data, 1) = 999	data becomes {100, 999, 300}
5	Print saved value and array element	40 and 999

Exact output:

```
Scaled Max: 40
Updated Array[1]: 999
Scaled Value Again: 40
```

Question 2: Static Data Member Linkage and Member Function Scoping

Errors and explanation

- Line 151 – Linker error: totalBankReserves is declared in the class but, in the traditional non-inline static-member model used here, it also needs one out-of-class definition to allocate storage.
- Line 163 – Compilation error: displayAudit is static and has no this pointer. It cannot access accHolder, which belongs to a particular BankAccount object. Pass an object or use a non-static member function to display account-specific data.

Corrected complete C++ code

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves;

public:
    BankAccount(std::string name, double initialDeposit)
        : accHolder(name), balance(initialDeposit) {
        totalBankReserves += initialDeposit;
    }

    void displayAccount() const {
        std::cout << "Account Holder: " << accHolder
                  << ", Balance: $" << balance << std::endl;
    }

    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $"
                  << totalBankReserves << std::endl;
    }
};

// Required definition of the static data member
double BankAccount::totalBankReserves = 0.0;

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    acc1.displayAccount();
    acc2.displayAccount();
    BankAccount::displayAudit();
    return 0;
}
```

Dry run

Step	Action	totalBankReserves
1	Static member initialized	0
2	Construct Alice with $5,000	5,000
3	Construct Bob with $3,000	8,000
4	Display each object, then class audit	8,000

Exact output:

```
Account Holder: Alice, Balance: $5000
Account Holder: Bob, Balance: $3000
Total Reserves in Vault: $8000
```

Question 3: Function Overload Ambiguity via Default Arguments

Errors and explanation

- Line 219 – Compilation error: computeVolume(5.0) matches both computeVolume(double, double = 1.0, double = 1.0) and computeVolume(double). Both are viable with one argument, so the call is ambiguous.
- Line 222 – The two-argument call can also be ambiguous: the cuboid overload can use its default height, while the cylinder overload takes exactly two arguments. Both candidates receive equally good conversions.
- Function signatures are determined by parameter types and count, not by default-argument values. A default argument does not create a separate overload.

Corrected design and complete C++ code

To remove ambiguity completely, give the three geometric operations distinct names. This preserves flexible cuboid dimensions and keeps the cylinder calculation clear.

```
#include <iostream>

double computeCuboidVolume(double length,
                           double width = 1.0,
                           double height = 1.0) {
    return length * width * height;
}

double computeCubeVolume(double side) {
    return side * side * side;
}

double computeCylinderVolume(double radius, double height) {
    const double PI = 3.14159;
    return PI * radius * radius * height;
}

int main() {
    std::cout << "Cube: " << computeCubeVolume(5.0) << std::endl;
    std::cout << "Cuboid: " << computeCuboidVolume(5.0, 2.0, 3.0)
              << std::endl;
    std::cout << "Cylinder: " << computeCylinderVolume(3.0, 7.0)
              << std::endl;
    return 0;
}
```

Dry run

Call	Calculation	Result
computeCubeVolume(5.0)	5 × 5 × 5	125
computeCuboidVolume(5, 2, 3)	5 × 2 × 3	30
computeCylinderVolume(3, 7)	3.14159 × 3 × 3 × 7	197.92017

Exact output:

```
Cube: 125
Cuboid: 30
Cylinder: 197.92
```

Question 4: Array of Objects Missing Default Constructor

Errors and explanation

- Line 272 – Compilation error: Student batch[3] requires the compiler to construct three Student objects with no arguments. Once the programmer declares a parameterized constructor, the compiler does not automatically provide a default constructor.
- Technique A: define an explicit zero-argument constructor, so each array element can be created before values are assigned.
- Technique B: initialize the array directly with parameterized objects, for example Student batch[3] = { Student(101, "Aarav"), ... }; this does not require a default constructor.

Corrected complete C++ code – Technique A

```
#include <iostream>
#include <string>

class Student {
private:
   int rollNumber;
   std::string name;

public:
   Student() : rollNumber(0), name("") {}

   Student(int r, std::string n)
      : rollNumber(r), name(n) {}

   void display() const {
      std::cout << "Roll: " << rollNumber
            << ", Name: " << name << std::endl;
   }
};

int main() {
   Student batch[3];

   batch[0] = Student(101, "Aarav");
   batch[1] = Student(102, "Bhavna");
   batch[2] = Student(103, "Chirag");

   for (int i = 0; i < 3; ++i) {
      batch[i].display();
   }
   return 0;
}
```

Dry run

Index	Assignment	Stored object
0	Student(101, "Aarav")	Roll 101, Aarav
1	Student(102, "Bhavna")	Roll 102, Bhavna
2	Student(103, "Chirag")	Roll 103, Chirag

Exact output:

```
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5: Friend Function Declaration and Scope Resolution Flaws

Errors and explanation

- Line 320 – Compilation error: TelemetryAuditor is used in the friend declaration inside Engine before the compiler has seen its class declaration. Add class TelemetryAuditor; before Engine.
- Line 335 – Compilation error: alertThreshold is not a local variable or a member of Engine. The friend function must access it through the explicitly passed auditor object, a.alertThreshold.
- Line 348 – Compilation error: a friend function is a non-member function. It has no implicit this pointer and cannot be called as v8.inspectDiagnostics(...). Call inspectDiagnostics(v8, audit).
- Friendship grants access to private members but does not make the friend function a member of either class.

Corrected complete C++ code

```
#include <iostream>

class TelemetryAuditor; // Forward declaration

class Engine {
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}

    friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

class TelemetryAuditor {
private:
    int alertThreshold;

public:
    TelemetryAuditor(int threshold) : alertThreshold(threshold) {}

    friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

void inspectDiagnostics(Engine e, TelemetryAuditor a) {
    if (e.temperature > a.alertThreshold) {
        std::cout << "Warning: Engine overheat!" << std::endl;
    } else {
        std::cout << "Engine operating nominally at "
                  << e.rpm << " RPM." << std::endl;
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    inspectDiagnostics(v8, audit);
    return 0;
}
```

Dry run

Value	Meaning	Check
105	Engine temperature	105 > 100 is true
100	Auditor alert threshold	Comparison limit
3200	Engine RPM	Used in nominal message only

Exact output:

```
Warning: Engine overheat
```

Section B – Pseudo-Code Writing Questions

Question 1: Corporate Vault Management with Static Data and Functions

Design overview

- Each BranchOffice object stores its own branch ID, name, and balance as instance attributes.
- totalVaultReserves and totalBranchesActive are STATIC class attributes shared by every branch.
- Deposits update both the branch balance and global reserves. A successful withdrawal reduces both; an invalid withdrawal changes neither.
- The question does not provide opening deposits. For a verifiable example, assume Branch1 opens with $500,000 and Branch2 with $300,000.

Structured pseudo-code

```
CLASS BranchOffice
PRIVATE:
  branchID : STRING
  branchName : STRING
  currentBalance : REAL
  STATIC totalVaultReserves : REAL ← 0.0
  STATIC totalBranchesActive : INTEGER ← 0

PUBLIC:
  CONSTRUCTOR(id : STRING, name : STRING, openingDeposit : REAL)
    branchID ← id
    branchName ← name
    currentBalance ← openingDeposit
    totalVaultReserves ← totalVaultReserves + openingDeposit
    totalBranchesActive ← totalBranchesActive + 1
  END CONSTRUCTOR

  METHOD Deposit(amount : REAL)
    IF amount > 0 THEN
      currentBalance ← currentBalance + amount
      totalVaultReserves ← totalVaultReserves + amount
    ELSE
      DISPLAY "Invalid deposit"
    ENDIF
  END METHOD

  METHOD Withdraw(amount : REAL) RETURNS BOOLEAN
    IF amount > 0 AND amount <= currentBalance THEN
      currentBalance ← currentBalance - amount
      totalVaultReserves ← totalVaultReserves - amount
      RETURN TRUE
    ELSE
      RETURN FALSE
    ENDIF
  END METHOD

  STATIC METHOD GetVaultSummary()
    DISPLAY "Active branches: ", totalBranchesActive
    DISPLAY "Total reserves: $", totalVaultReserves
  END METHOD

  STATIC METHOD GetGlobalLiquidityRatio(targetReserve : REAL) RETURNS REAL
    IF targetReserve <= 0 THEN
      RETURN 0
    ENDIF
    RETURN totalVaultReserves / targetReserve
  END STATIC METHOD
ENDCLASS
```

```
BEGIN MAIN
  DECLARE Branch1, Branch2 : BranchOffice
  DECLARE withdrawalStatus : BOOLEAN
  DECLARE liquidityRatio : REAL

  Branch1.Initialize("B01", "Central", 500000)
  Branch2.Initialize("B02", "North", 300000)

  Branch1.Deposit(150000)
  withdrawalStatus ← Branch2.Withdraw(50000)

  BranchOffice.GetVaultSummary()
  liquidityRatio ← BranchOffice.GetGlobalLiquidityRatio(1000000)
  DISPLAY "Liquidity ratio: ", liquidityRatio
END MAIN
```

Note: Initialize in the driver represents construction with the stated constructor arguments.

Dry-run trace

Stage	Branch1 balance	Branch2 balance	Total vault reserves	Active branches
Initial state	$500,000	$300,000	$800,000	2
Branch1 deposits $150k	$650,000	$300,000	$950,000	2
Branch2 withdraws $50k	$650,000	$250,000	$900,000	2
Liquidity ratio (target $1m)	—	—	0.90	2

Final summary: 2 active branches; consolidated reserves = $900,000; liquidity ratio against $1,000,000 target = 0.90.

Question 2: Matrix Mathematical Engine and Friend Dot-Product

Design overview

- Matrix2D encapsulates a fixed 3 × 3 REAL array.
- All member methods are declared inside the class and defined outside it using Matrix2D::MethodName().
- SetElement and GetElement validate row and column bounds.
- ComputeCrossTrace is a non-member friend; it directly reads the private diagonal values of both objects and computes their dot product.

Structured pseudo-code

```
CLASS Matrix2D
PRIVATE:
  grid : ARRAY[3][3] OF REAL

PUBLIC:
  DECLARE METHOD Initialize()
  DECLARE METHOD SetElement(row : INTEGER, col : INTEGER, value : REAL)
  DECLARE METHOD GetElement(row : INTEGER, col : INTEGER) RETURNS REAL
  DECLARE METHOD Display()
  FRIEND FUNCTION ComputeCrossTrace(A : Matrix2D, B : Matrix2D) RETURNS REAL
ENDCLASS

METHOD Matrix2D::Initialize()
  FOR row ← 0 TO 2
    FOR col ← 0 TO 2
      grid[row][col] ← 0.0
    ENDFOR
  ENDFOR
END METHOD
```

```
METHOD Matrix2D::SetElement(row : INTEGER, col : INTEGER, value : REAL)
  IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
    grid[row][col] ← value
  ELSE
    DISPLAY "ERROR: Matrix index out of bounds"
  ENDIF
END METHOD

METHOD Matrix2D::GetElement(row : INTEGER, col : INTEGER) RETURNS REAL
  IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
    RETURN grid[row][col]
  ELSE
    DISPLAY "ERROR: Matrix index out of bounds"
    RETURN 0.0
  ENDIF
END METHOD

METHOD Matrix2D::Display()
  FOR row ← 0 TO 2
    FOR col ← 0 TO 2
      DISPLAY grid[row][col], " "
    ENDFOR
    DISPLAY NEWLINE
  ENDFOR
END METHOD

FUNCTION ComputeCrossTrace(A : Matrix2D, B : Matrix2D) RETURNS REAL
  DECLARE total : REAL ← 0.0
  FOR i ← 0 TO 2
    total ← total + (A.grid[i][i] * B.grid[i][i])
  ENDFOR
  RETURN total
END FUNCTION

BEGIN MAIN
  DECLARE A, B : Matrix2D
  A.Initialize()
  B.Initialize()

  A.SetElement(0, 0, 2)
  A.SetElement(1, 1, 4)
  A.SetElement(2, 2, 6)

  B.SetElement(0, 0, 1)
  B.SetElement(1, 1, 3)
  B.SetElement(2, 2, 5)

  DISPLAY "Cross trace: ", ComputeCrossTrace(A, B)
END MAIN
```

Dry-run trace

i	A[i][i]	B[i][i]	Product	Running total
0	2	1	2 × 1 = 2	2
1	4	3	4 × 3 = 12	14
2	6	5	6 × 5 = 30	44

Final answer: ComputeCrossTrace(A, B) = 44.

Question 3: Student Academic Roster Using Array of Objects

Design overview

- Create three StudentRecord objects in an array; the default constructor initializes each object safely.

- ReadData stores roll number, name, and three marks, then calls the private CalculateResult method.
- Percentage is the average of the three subject marks. Grade thresholds are A ≥ 85, B ≥ 70, C ≥ 50, otherwise F.
- Use Bubble Sort in descending order of percentage. Since sample marks are not specified, the trace uses clearly stated sample records.

Structured pseudo-code

```
CLASS StudentRecord
PRIVATE:
  rollNo : INTEGER
  studentName : STRING
  marks : ARRAY[3] OF REAL
  percentage : REAL
  grade : CHARACTER

  METHOD CalculateResult()
    percentage ← (marks[0] + marks[1] + marks[2]) / 3
    IF percentage >= 85 THEN
      grade ← 'A'
    ELSE IF percentage >= 70 THEN
      grade ← 'B'
    ELSE IF percentage >= 50 THEN
      grade ← 'C'
    ELSE
      grade ← 'F'
    ENDIF
  END METHOD

PUBLIC:
  CONSTRUCTOR()
    rollNo ← 0
    studentName ← ""
    marks[0] ← 0
    marks[1] ← 0
    marks[2] ← 0
    percentage ← 0
    grade ← 'F'
  END CONSTRUCTOR

  METHOD ReadData(r : INTEGER, name : STRING,
            m1 : REAL, m2 : REAL, m3 : REAL)
    rollNo ← r
    studentName ← name
    marks[0] ← m1
    marks[1] ← m2
    marks[2] ← m3
    CALL CalculateResult()
  END METHOD

  METHOD GetPercentage() RETURNS REAL
    RETURN percentage
  END METHOD

  METHOD Display()
    DISPLAY rollNo, " ", studentName, " ",
          percentage, "% Grade ", grade
  END METHOD
ENDCLASS

BEGIN MAIN
  DECLARE students : ARRAY[3] OF StudentRecord
  DECLARE i, j : INTEGER
  DECLARE temp : StudentRecord

  students[0].ReadData(101, "Aarav", 70, 80, 75)
```

```
    students[1].ReadData(102, "Bhavna", 90, 95, 85)
    students[2].ReadData(103, "Chirag", 60, 65, 70)

    FOR i ← 0 TO 1
      FOR j ← 0 TO 1 - i
        IF students[j].GetPercentage() <
          students[j + 1].GetPercentage() THEN
          temp ← students[j]
          students[j] ← students[j + 1]
          students[j + 1] ← temp
        ENDIF
      ENDFOR
    ENDFOR

    DISPLAY "Rank List"
    FOR i ← 0 TO 2
      DISPLAY "Rank ", i + 1
      students[i].Display()
    ENDFOR
  END MAIN
```

Sample percentage calculations

Student	Marks	Percentage	Grade
Aarav	70, 80, 75	75%	B
Bhavna	90, 95, 85	90%	A
Chirag	60, 65, 70	65%	C

Bubble-sort dry run

Pass / comparison	Compared percentages	Action	Array order after action
Initial	—	—	Aarav 75, Bhavna 90, Chirag 65
Pass 1, j=0	75 < 90	Swap	Bhavna 90, Aarav 75, Chirag 65
Pass 1, j=1	75 < 65	No swap	Bhavna 90, Aarav 75, Chirag 65
Pass 2, j=0	90 < 75	No swap	Bhavna 90, Aarav 75, Chirag 65

Final descending rank: 1. Bhavna – 90% (A); 2. Aarav – 75% (B); 3. Chirag – 65% (C).

Question 4: Flexible Logger with Overloaded Functions and Inline Helpers

Design overview

- FormatPrefix is an inline helper that returns a tag in square brackets followed by a space.
- The source signatures contain an ambiguity: LogEvent(message) and LogEvent(message, severityLevel = 1, timestamp = ...) are both viable for a one-argument call.
- To preserve a one-argument INFO call and the severity-based overload, remove the default value for severityLevel but retain the default timestamp. This makes one-argument, two-argument, and three-argument calls unambiguous.
- The hardware-fault overload is distinguished by its parameter types (INTEGER, STRING).

Corrected structured pseudo-code

```
  INLINE FUNCTION FormatPrefix(prefixTag : STRING) RETURNS STRING
    RETURN "[" + prefixTag + "] "
  END FUNCTION

  OVERLOAD 1:
  FUNCTION LogEvent(message : STRING)
    DISPLAY FormatPrefix("INFO") + message
```

```
END FUNCTION

OVERLOAD 2:
FUNCTION LogEvent(message : STRING,
                  severityLevel : INTEGER,
                  timestamp : STRING = "00:00:00")
   DECLARE severityName : STRING
   IF severityLevel = 1 THEN
      severityName ← "LOW"
   ELSE IF severityLevel = 2 THEN
      severityName ← "MEDIUM"
   ELSE IF severityLevel = 3 THEN
      severityName ← "CRITICAL"
   ELSE
      severityName ← "UNKNOWN"
   ENDIF
   DISPLAY "[" + severityName + "] @ " + timestamp + " : " + message
END FUNCTION

OVERLOAD 3:
FUNCTION LogEvent(errorCode : INTEGER, systemModule : STRING)
   DISPLAY "[HARDWARE_FAULT] Module: " + systemModule +
         " -> Error Code: 0x" + HEX(errorCode)
END FUNCTION

BEGIN MAIN
   LogEvent("System started")
   LogEvent("Temperature rising", 2)
   LogEvent("Pressure critical", 3, "14:30:00")
   LogEvent(26, "SENSOR")
END MAIN
```

Call-resolution trace

Client call	Selected overload	Default argument used	Resulting format
LogEvent("System started")	Overload 1	None	[INFO] System started
LogEvent("Temperature rising", 2)	Overload 2	timestamp = 00:00:00	[MEDIUM] @ 00:00:00 : Temperature rising
LogEvent("Pressure critical", 3, "14:30:00")	Overload 2	None	[CRITICAL] @ 14:30:00 : Pressure critical
LogEvent(26, "SENSOR")	Overload 3	None	[HARDWARE_FAULT] Module: SENSOR -> Error Code: 0x1A

Important exam point: default arguments are applied after overload selection; they do not create a distinct overload. Avoid overlapping callable signatures.

Question 5: Hardware Register Bank with Return-by-Reference

Design overview

- RegisterBank stores eight integer register values and one errorSink value.
- GetRegister returns REF INTEGER, so the caller receives an alias to an existing stored integer.
- For a valid index, the returned reference aliases registers[index]. For an invalid index, it aliases errorSink and raises an alert.
- Because the returned reference designates a modifiable lvalue, a call such as GetRegister(0) <- 170 can appear on the left side of assignment.

Structured pseudo-code

```
CLASS RegisterBank
PRIVATE:
```

```
    registers : ARRAY[8] OF INTEGER
    errorSink : INTEGER

  PUBLIC:
    CONSTRUCTOR()
      FOR i ← 0 TO 7
        registers[i] ← 0
      ENDFOR
      errorSink ← -1
    END CONSTRUCTOR

    METHOD GetRegister(index : INTEGER) RETURNS REF INTEGER
      IF index >= 0 AND index < 8 THEN
        RETURN REF registers[index]
      ELSE
        DISPLAY "ALERT: Register index out of bounds"
        RETURN REF errorSink
      ENDIF
    END METHOD

    METHOD DisplayState()
      DISPLAY "Register Bank:"
      FOR i ← 0 TO 7
        DISPLAY "R", i, " = 0x", HEX(registers[i])
      ENDFOR
    END METHOD
  ENDCLASS

  BEGIN MAIN
    DECLARE bank : RegisterBank
    DECLARE readVal : INTEGER

    bank.GetRegister(0) ← 170
    bank.GetRegister(1) ← 255

    readVal ← bank.GetRegister(0)
    DISPLAY "Read Register 0: ", readVal

    bank.GetRegister(10) ← 999
    DISPLAY "Error sink: ", bank.GetRegister(10)

    bank.DisplayState()
  END MAIN
```

Dry-run trace

Step	Operation	Register state / result
1	Constructor	R0–R7 = 0; errorSink = -1
2	GetRegister(0) ← 170	R0 = 170 = 0xAA
3	GetRegister(1) ← 255	R1 = 255 = 0xFF
4	readVal ← GetRegister(0)	readVal = 170
5	GetRegister(10) ← 999	Alert displayed; errorSink becomes 999; register array unchanged
6	DisplayState	R0=0xAA, R1=0xFF, R2–R7=0x0

Return-by-reference explanation: a function declared to return REF INTEGER returns an alias to an existing INTEGER object, not a copy. The expression bank.GetRegister(0) is therefore an lvalue referring to registers[0], so assignment writes directly into that array element. The returned object must remain alive while the reference is used; returning a reference to a local temporary would create a dangling reference.

Quick Revision

Debugging – key C++ rules

Topic	Exam-ready point
Return by reference	Return references only to objects that remain alive after the function returns; local automatic variables must be returned by value.
Array element reference	A reference to a caller-owned array element is valid while the array and element remain alive and the index is valid.
Static data member	A traditional static data member needs one out-of-class definition to provide storage.
Static member function	Has no this pointer; it can directly access only static members. Use an object for instance members.
Default arguments	Applied after overload selection; overlapping defaults can make calls ambiguous.
Overloading	Overloads must differ by parameter list; return type alone cannot distinguish them.
Default constructor	A user-declared parameterized constructor means the compiler does not automatically generate a zero-argument constructor.
Friend function	A non-member function with granted private access; call it like a normal function and pass objects explicitly.
Forward declaration	Declare a class name before using it in another class declaration when the complete definition is not yet available.

OOP pseudo-code – key patterns

Topic	Remember
Static class state	Declare STATIC attributes once at class level; all objects share the same values.
Scope resolution	Use ClassName::MethodName() for member methods defined outside the class.
Friend function	Declare FRIEND FUNCTION in the class; use explicit object parameters to access private data.
Array of objects	Provide a default constructor when declaring an uninitialized fixed-size object array.
Bubble sort	Compare adjacent percentages and swap when the left value is smaller for descending order.
Inline helper	Use for a small formatting operation; it is a request to the compiler, not a guarantee of expansion.
Default arguments	Place defaults on trailing parameters; avoid a one-argument overload that overlaps with a defaulted overload.
Return by reference	Return REF to a persistent array element or member; assignment through the call changes that stored element.

Verified numerical answers

Question	Final result
A1	Scaled maximum = 40; updated array element = 999
A2	Alice $5,000; Bob $3,000; total reserves $8,000
A3	Cube 125; cuboid 30; cylinder 197.92
A4	Three Student objects display roll numbers 101, 102, 103
A5	Temperature 105 > threshold 100; overheat warning
B1	Assumed opening reserves $800,000; after +$150k and −$50k = $900,000; ratio 0.90

B2	Diagonal dot product = 2 + 12 + 30 = 44
B3	Sorted percentages: 90%, 75%, 65% (Bhavna, Aarav, Chirag)
B4	One-arg INFO; two-arg MEDIUM with default time; three-arg CRITICAL with supplied time; hardware code 0x1A
B5	R0 = 0xAA; R1 = 0xFF; invalid index redirects assignment to errorSink